Microsoft Foundry

VOLT & CO / INTERNAL ENABLEMENT

From useful agent to governed service

Microsoft Foundry + Citadel

One customer journey. Two learning loops.

80 minutes of content + 10 minutes of Q&A



Two loops. One improving service.

Build and prove in DEV. Release and learn in production-like DEMO.

BRIDGE 1 · HANDOFF

DEV → CANDIDATE + EVIDENCE → DEMO

Code · tools · knowledge · tests · policy

INNER LOOP · BUILD & PROVE

1. **Build & connect**
   Connect tools + policy.
2. **Run, debug & trace**
   Inspect one execution.
3. **Evaluate & red-team**
   Test correctness + safety.
4. **Guard & optimize**
   Compare held-out cases.

SHARED CONTRACT & GATES

Same repo
Same tests
Same safety
Separate IDs

OUTER LOOP · RELEASE & LEARN

5. **Gate & release**
   Gate the exact candidate.
6. **Prove readiness**
   Verify the serving version.
7. **Operate & monitor**
   Watch requests + failures.
8. **Learn & improve**
   Review the next test case.

BRIDGE 2 · FEEDBACK

DEMO → HUMAN REVIEW → DEV

Reviewed traces become new regression tests.

Source workshop lifecycle diagram · adapted for synthetic DEV / DEMO

One customer. One support journey.

Volt Charge 65 / batch CH-0722 / order VC-10482

SYNTHETIC CUSTOMER CASE

“Can I get a replacement?”

Outside the standard 30-day return window.

A recall and a return window are not the same decision.

1 **Establish the facts**

Recall, order and eligibility tools

2 **Ground the policy**

Foundry IQ returns a source citation

3 **Choose a responsible next step**

Human assessment—not an automatic refund

Choose the operating model

Use the managed path unless the application needs custom runtime control.

The decision	Prompt agent	Hosted agent
Authoring	Instructions and configuration	Framework or custom code
Runtime responsibility	Managed orchestration	Your logic; managed hosting
Choose it when…	The managed path is enough	You need deterministic logic
Cost shape	Model and tool usage	Usage + active compute

VOLT'S CHOICE Agent Framework + Responses → deterministic decisions and lifecycle integration

Our code. Foundry's runtime.

Hosted is the control path—not the unmanaged path.

YOU OWN

Agent logic + orchestration

Framework + dependencies

Responses request handling

Configuration + sizing

FOUNDRY PROVIDES

Managed hosting + endpoint

Scaling + agent identity

Tracing + evaluations

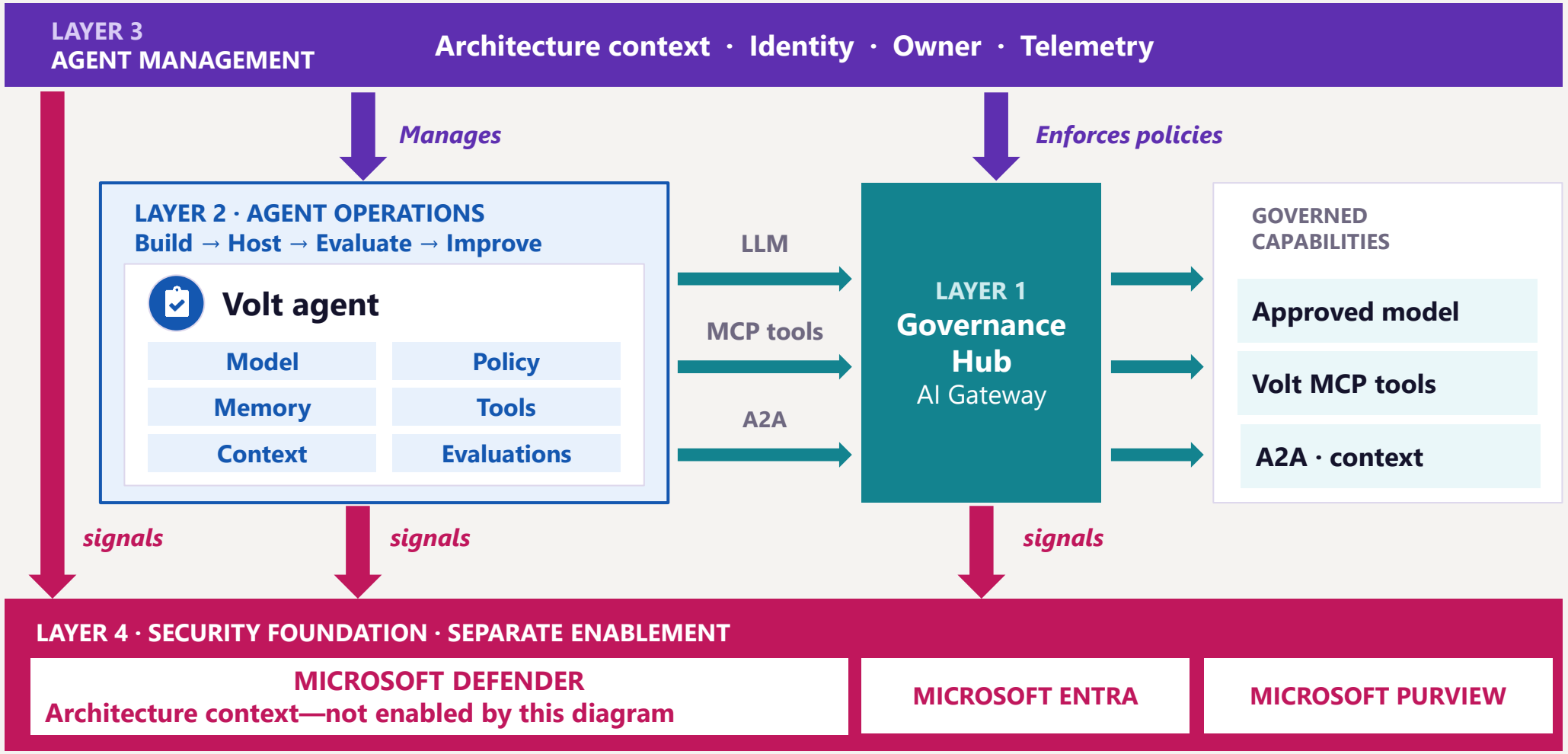
Operate + monitor

GitHub Copilot helps us develop the application.

It is not the deployed runtime.

Where Citadel and Foundry meet

Focus today: governed model/tool access + agent operations. Other layers are architecture context.



Give the agent governed capabilities

Tools → a curated Toolbox → a pinned agent version

CONCEPTUAL ROUTE · NOT A PORTAL SCREENSHOT



One answer we can inspect—not an unrestricted tool catalog.

Ground the answer in policy

Foundry IQ connects source evidence to the customer response.

1 Sources

Replacement policy

Synthetic recall notice

Azure AI Search

2 Knowledge base

volt-support-knowledge

Text + semantic extractive retrieval

3 Agent + citation

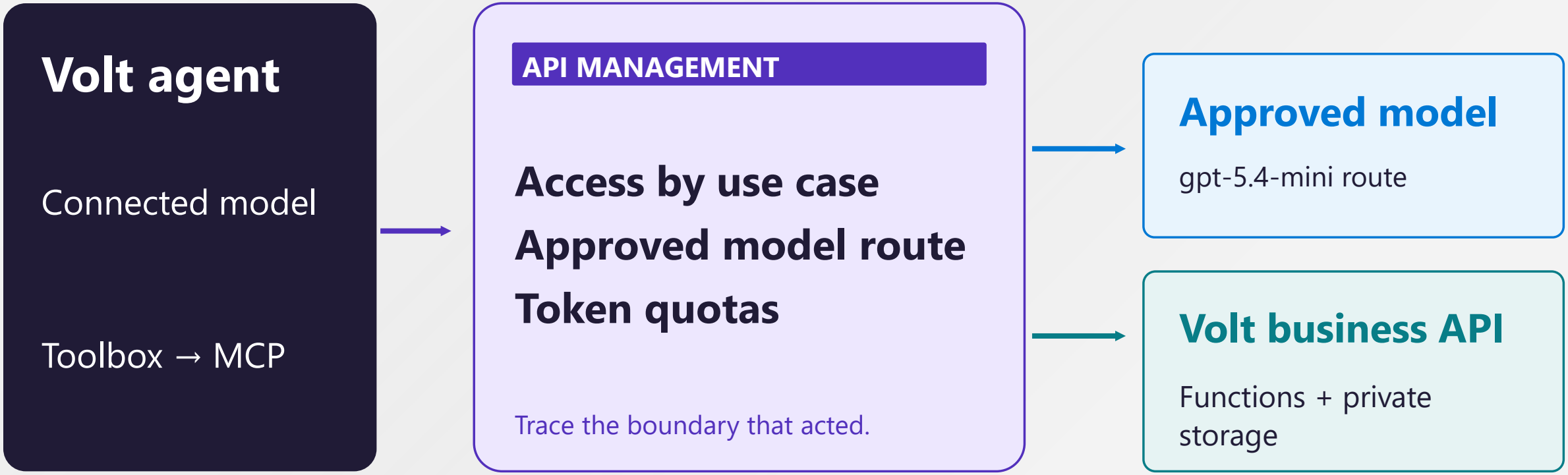
Retrieve evidence.
Keep the source.
Explain next steps.

A citation is inspectable.

Policy knowledge complements business tools. It does not replace them.

One gateway. Explicit access contracts.

Citadel's Governance Hub governs the calls routed through it.



An access contract is not customer order-ownership authorization.

Enforce the right boundary

Authorization ≠ content guardrails ≠ business eligibility

1

Input

Inspect content at supported boundaries.

2

Reason

Plan from instructions and context.

3

Tool access

Check the consumer's access contract.

4

Business rule

Apply eligibility and safety rules.

5

Response

Inspect the outcome and its evidence.

Prove permitted utility and the specific block—not just a refusal.

Remember preferences, not authority

The same caller can return. The facts still come from authoritative systems.

A **Caller A · session 1** →

Write the preference

A **Caller A · session 2** ←

Return the saved preference

VOLT-PREFERENCES

“Concise answers. Next step first.”

User profile only
7-day retention
Explicit forget

B **Caller B**

Separate authenticated scope—not a new chat for Caller A.

NO SHARED PREFERENCE

Keep policy, permissions and customer facts out of preference memory.

Evidence before confidence

A fluent answer can still make the wrong business decision.

“It sounds right.”

Coherence

Useful quality evidence.
Not recall correctness.

BUSINESS CONTRACT

Structured decision

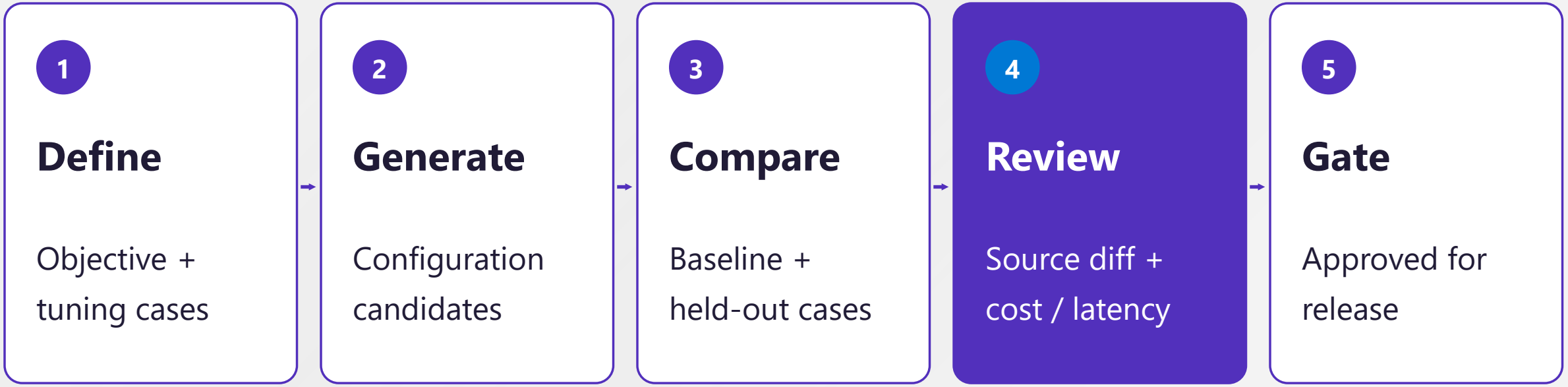
+ observed tool evidence

+ policy citation when required

Compare exact versions on reviewed cases. Keep tuning data separate from holdouts.

Optimize configuration. Review the change.

The best score starts a review—it does not authorize a release.

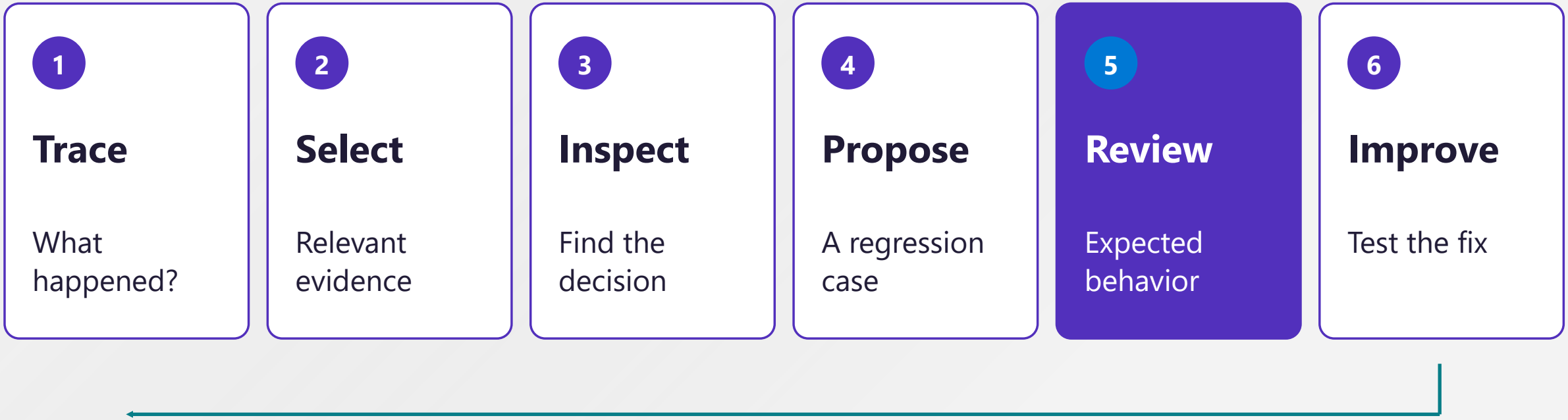


A code defect needs a code fix. An optimizer candidate can also be rejected.

Operation teaches DEV what to test

Tool result: eligible. Agent decision: recommendation.

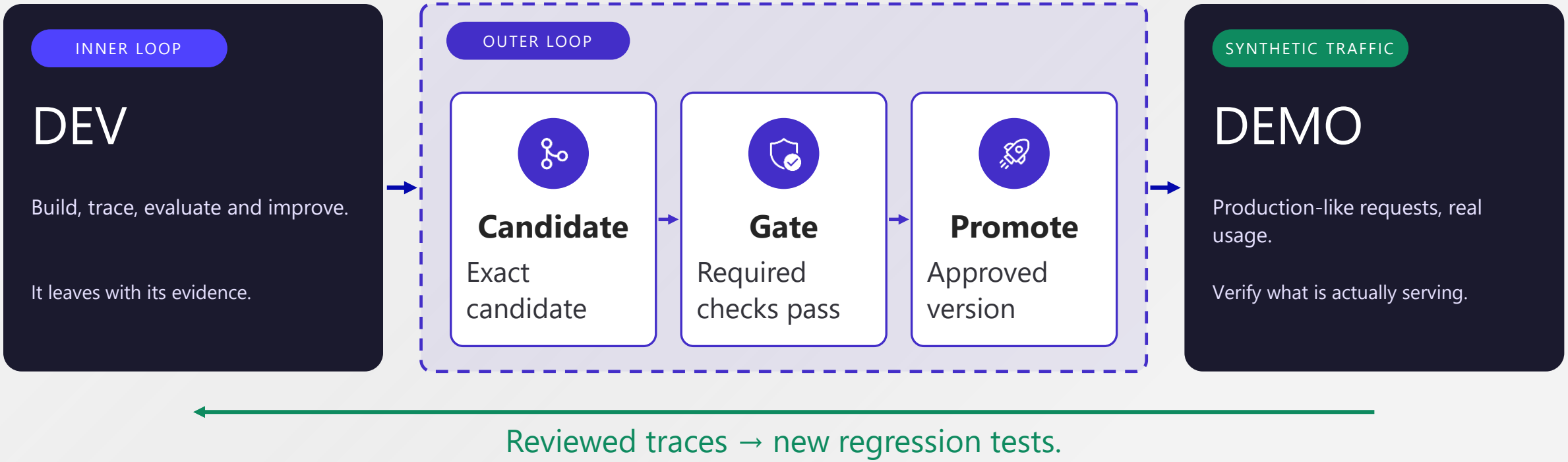
OBSERVED BUSINESS-CONTRACT MISMATCH



A trace is evidence. A person supplies the reviewed expectation.

A PR carries code and behavioral evidence

One reviewed candidate. Separate environments. A verifiable serving version.

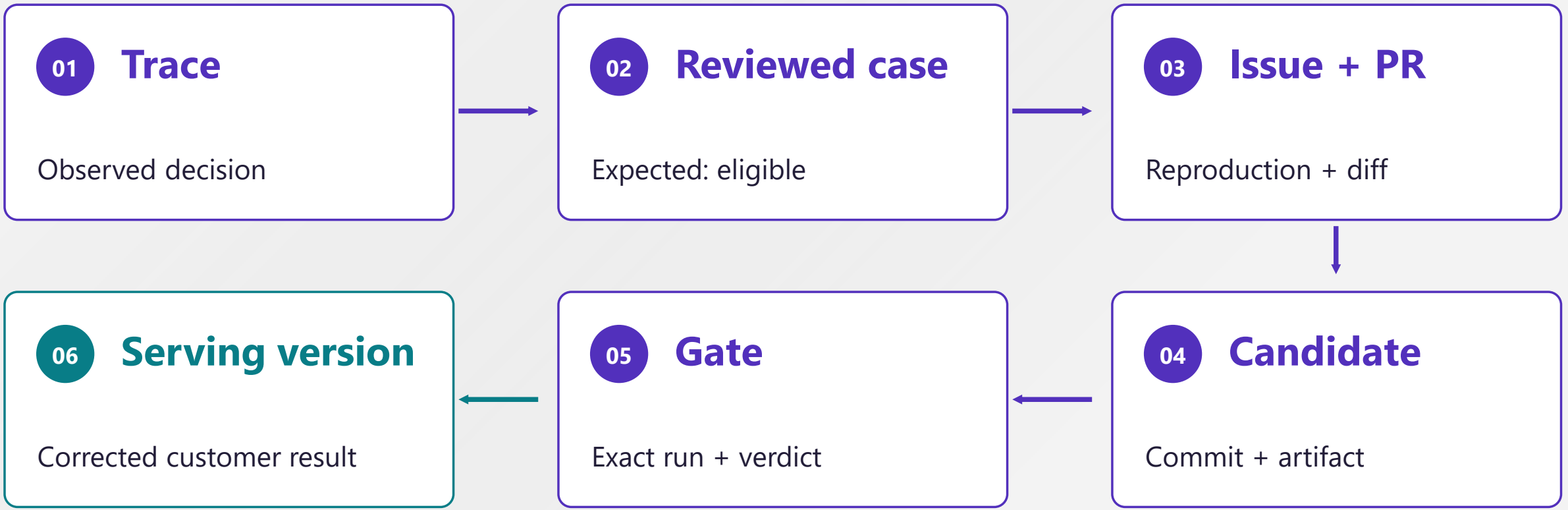


A passing PR is not the finish line. Verify the serving version and the customer outcome.

DEMO is a synthetic environment—not customer production.

Follow one fix from trace to customer

Every link should identify the same change—not merely the latest version.



Version identity + behavioral evidence + human judgment

Useful. Governed. Measurable. Improving.

Four ideas to take back to your next agent.

1 Useful

Tools establish facts.
IQ supplies policy evidence.

2 Governed

Explicit access and boundaries.
Memory stays deliberately narrow.

3 Measurable

Correctness is a contract.
Quality is more than fluency.

4 Improving

Reviewed traces become tests.
Gated changes reach customers.

10 MINUTES FOR YOUR QUESTIONS